DURIAUD Thomas - TP095818
BUCLON Mathis – TP095803

Machine Learning

Project

Categorical classification of video games on giant Steam dataset

Group Assignment

Abstract

For illustrate our competences learning during Intro to Machine Learning courses, and make them in a real case, we have chosen to study a very interesting problem: to determine a video game's category from its data. With metadata on video games, some online platforms like Steam, Epic Games, or others can classify the new games they sales by algorithms. And more, this permits us to study very interesting and complex problems, try some methods, go back, try other, see the problems in the data, adapt them... So, all of this illustrates and makes into practice our Machine Learning courses, and we delve into the fascinating field of algorithmic data analysis.

Firstly, of course we have done preprocessing on our data. After an initial cleaning, we have built more interesting and advanced cleaning functions linked to our dataset.
After, we have a step of further cleaning and pre-label encoding. In this, we use the function building just before and pre-label some columns for prepare our strategy of Multiclass.

Finaly, we split data between train and test data and encoding them. All that remained was to do class balancing on some columns and feature scaling on some others, and the dataset was ready to train an algorithm.
We have trained two different algorithms: a first step to have a solid foundation, which we have seen in class and whose interpretability and comprehension are good, KNN (K-nearest neighbours). And a second better suited to our project, recognized for its ability to manage many values and find even weak correlations, LightGBM.
Finally, we evaluated our algorithms with many widely used metrics: accuracy, recall, F1-score and precision. And for better understand our results, improve our models, we generated confusion matrix, validation and learning curves, some accuracy and recall graphics …

1 - Introduction

Our problem can be summed up in one sentence: find the category of a video game based on additional metadata such as kind of the video game, platforms, score rank...

So, in the case of our courses, design and evaluate a machine learning model capable of accurately predicting the category of a video game based on structured metadata.
At first glance, this can be non-intuitive. But there exist relationships between these features, particularly suitable for a machine learning approach. Instead, with 'train' data, an algorithm can understand, learn and reproduce the relationships between features in a set of 'test' data.

Video game classification is an important problem for digital distribution platforms such as Steam, as it enables better organization of large catalog, improved recommendation systems, and more efficient content discovery for users. Given the rapid growth of the video game industry and the increasing volume of available data, automated classification methods have become essential.

Firstly, regarding the dataset. We must find data that can be used for training and testing an algorithm. That is to say, the data must be recent, complete and usable for a problem like ours.

As this problem is well established and of practical interest. So, there are many datasets on the web that contain data like we want. To follow what a specialist engineer would have done, we are going to search for data on specialized websites like Kaggle. And from this place, we reviewed many dataset pages to find what we wanted: a complete and reliable dataset.

So, we have decided to take this one:
(ref: https://www.kaggle.com/datasets/fronkongames/steam-games-dataset/data)

Now we have the data, and the technical part of our problem begins. Because we can't give raw data to an algorithm, significant preprocessing steps are required, including data cleaning, feature selection, normalization, and encoding of categorical variables.

For example, let's take data of the category columns. In the original dataset there is one column with multiples category for each game, so we must find a solution for making this data readable for the algorithm. That's to say, manage both to separate the different categories (to make it machine-readable) and to control their correlation (to make it machine efficient)

Instead, some features of the data set are unusable for our project, or some others need to be formatted, or simplified so that the algorithm has only useful data. This is for the purpose of improving algorithm performance, its response time and success of the process.

To briefly outline our approach to the problem we have decided to formulate as a multiclass classification task with a limited number of classes, for data like category, kind of game. This will be explained in detail later. This constituted one of the main challenges of the project. But we met others in the reading, formatting, and standardization of some data, requiring advanced cleaning functions.

Another challenge we encountered in this project is the imbalance and distribution for target classes (which significantly affect classification accuracy).

Furthermore, finding the most appropriate classification algorithm and its optimal hyperparameters require systematic experimentation and evaluation. Notably with a lot of data, a lot of features, having the best result requires a significant number of trials, which may involve iterative reassessment of modelling choice.

2 - Related works

This section reviews previous work related to game classification and multiclass learning on large-scale. We focus on studies using video game metadata as well as on machine learning methods relevant to our experimental setup.

1 – Direct applications and realization of Machine Learning works

Many projects exist on dataset equivalent at our (but never the same). Proof that the subject is interesting, thought-provoking, and interestingly complex compared to the different machine learning algorithm existing. Are we heading towards a multi-class system? Or multi-label? And which algorithm to use for this problem? Some researchers have used similar algorithms to us; thus, we can find some paper with detailed explanation of the models we use, but not the same way and without the same dataset. This work [1] starting from a dataset considerably smaller than ours try seven models on it; including LightGBM which turns out to be the most efficient (with an accuracy of 88,17%). But it's important to note this project and try to classify the game into only two categories (popularity positive/negative). And this paper teaches us that for this problem, a tree model like LightGBM or XGboost is very competitive. Of course, it's not the best algorithm existing, or rather, there are several approaches to our problem.

For example, another paper [2] proposes a multimodal deep learning-based approach to classify video game genres from images and descriptive texts, using CNNs for visuals and text encoders for descriptions. This approach teaches automatically complexes representations by raw data, contrary to our model which is based on tabular and structured features and being analysed with simpler algorithms (like LightGBM or KNN). Thus, the CNN model captures patterns that are more complicated to discern, which are not explicitly represented in classic data vectors. But a model like CNN requires a lot of power and time to operate and does not work well on data with too much noise.

In itself, it is always useful to test a large variety of algorithms to find the best fit for our problem. And it is with this in mind that the project [3] gives an analysis of many algorithms categories, with classical algorithm (SVM, Decision Trees), tree algorithm, boosting tree and others.

But now we are here, let's take a moment to discuss the project [4] because it presents a very interesting observation: a tree algorithm like RandomForest is less powerful than XGBoost or LightGBM. And this is shown in accuracy, F1-score and variance between train and test.

Why? Because each tree is trained independently on a bootstrapped subset. So, the final prediction is a mean or a vote of the trees. Thus, errors are not corrected, and RandomForest is sensitive to majority classes. (by overfitting majority classes)

Conversely, in boosting algorithms like LightGBM the learning is sequential: each tree is trained to correct the errors of the previous one. And more, the boosting

permits to capture nonlinear interactions and signals buried in noise. All of this makes a boosting algorithm much more powerful on a dataset like ours, because there is a lot of noise in the data, and certain patterns are lost in the dominant flow.

But let's return to our previous study [3]. The big benchmark isn't the only thing to retain of the paper. First, the benchmark uses known metrics used in our model: F1-score, accuracy and recall. Secondly, it tests the models under several conditions with the same dataset: with content descriptor of the games only, with text features only and all combined. Overall, the XGBoost model is the best, but closely followed by GradientBoosting, model representing the lowest level of boosting algorithm. And in particular, this explains why the dataset contains complex non-linear relationships, based on interactions between variables. We view clearly see a trend for boosting algorithms in our case of study. But is it true for all types of datasets (example, if we have fewer classification variables) or if we target other types of labels?

In the research, the paper [5] attempts to classify video games using a very simple, even counterintuitive, algorithm: Naive Bayes. This is one part of the principle that all features are independent conditional on the class. We know this affirmation is false, rarely true in real data. Notably in our case where, for example, critic scores are correlated to genre and game style. But with their probabilistic approach, Naive Bayle achieves satisfactory results: 71.8% accuracy and an F1 score of 70%.

And all of this by possessing very good interpretability. At questions like "Why is this game classified like this?" or "What general rules has the model learned?", Naive Bayes is good on the two because it based on an explicit probabilistic framework. Conversely, decision-making models like LightGBM with hundreds of trees, nonlinear nested interactions and without clear rules, it is impossible for a human to understand directly the model's operation. However, it’s important to note that a model like LightGBM has a lot of room for improvement, Naive Bayes is limited by its probabilistic conception.

In a more advanced form, there are many other algorithms that can solve our problem. And our problem is far from being the only useful analysis using this data. The project [6] wants another classification: based on a dataset like ours, by analysing historical video game sales data, it predicts their future sales and thus classifies them into two categories: high and low sales. The project detailed a step of preprocessing data like ours and using some algorithms of binary classification using regression. While SVR (Support Vector Regression) performs best, LR (Logistic Regression) offers the best compromise between performance and stability.

This choice of algorithms is very interesting because it differs greatly from your project, when we use a multiclass classification the [6] projects all the data onto a regression to perform a binary classification. The project shows another point: some models are subject to overfitting and link the importance with cross-

validation (for better stability between test/train). In our project, this is realized by hyperparametric tunning and stratified validation allowing us to have a very good fitting and stable model.

Although the models studied so far have all been predictive, other studies show a way to use machine learning to understand and explain. Instead, we know some machine learning models are specialized for finding links between data. In this way, this work [7] applies to machine learning technics for identifying principal factors that explain financial success of a video game. So here objective is not classification but predict and explain commercial success with some heterogeneous metrics coming from different platforms like Steam, Twitch and Metacritic. The model used isn't unique or standardized, but integrates tree regression, measure of correlation and importance of features. As in our project, the researcher trait processes video game data, builds models for extract useful patterns and combines several explanatory variables.

And notably, this model point correlation of feature like number of subscribers to a Steam page, peak concurrent player count on Twitch, and total number of reviews on Metacritic. He therefore discovers useful patterns enriched by data from heterogeneous sources, emphasizing the term of "success drivers" rather than only the predictive performance. (helping too how explain the functioning and interpretability of a model)

In contrary, other model want to find the same thing as our model (category of a video game), but not with the same features. Like this project [8] that describes an automatic genre classification of video game from App Store iOS, with only their title et description. And it is here that something interesting happens when in our project we are dropped the description columns (deemed very difficult to analyse), with the description researcher built a document-term matrix (each game is associated with a frequence of words). All this vector forms a very big matrix, reducing by LSI (Latent Semantic indexing) to a lower dimension latent space. Then with a classic supervised model SVM (Support Vector Machine) train with the reduced matrix and optimized with grid search method, via 20-fold cross-validation the best model achieves an accuracy of 77%. So, what's very interesting is to see how with only a text, a classification algorithm can work well; and the difference between our projects where we use mixed variables. The framework is therefore very different, but the goal is still same.

So just before, we have seen another methodology of classification only by text. In the columns we have dropped there is another one whose study is fascinating: screenshot. This column was also very difficult to study at our level. But the paper [9] speak about methodology for clustering video games by a collection of images. With 100k screens of 175 different video games, 10 genres of sport game. Here the researchers present a pre-training feature model that can later be used for classification.
The paper highlights the 'domain gap' concept: fact that same genre video gale can change a lot visually. For them, they explain the worst work of some methods

like ResNet and Fine-tuning which do not capture these changes well. But the paper highlights another approach, that of Contrastive Learning, that compare peers of examples instead of simple labels and obtain an accuracy of 91,83%. For better understanding how this works, Contrastive learning is a representational learning approach that seeks to bring similar examples (here, images of games of the same genre) closer together in the space of embeddings and to move different examples further apart. And after the train, the embedding visualization with t-SNE presents well-structured clusters, indicating that the model captures semantic similarities better than what could be seen with the naked eye. So, compared to our project focused on supervised tabular classification, the [9] paper offers another point for complex data treatment.

Now, why can't we use both textual and visual data? In this objective, the article [10] purpose to classify video games into different categories of age restriction, by using some pictures and description of 39212 video games. This subject is very useful because usually, this decision relies on human beings and is subjective and costly. Here it's proposed to automate this. And what's also interesting is that the test paper has 4 models: one text-based, one visual-based, and two with combination of text and image but not the same analysis method. The results directly demonstrate the importance of multimodality: combination of two different heterogeneous feature increase model capacity to detect sensitive content.
But one point is very bad with this method: interpretability. Because the combination of text and image data requires advanced fusion and a deep learning network. Compared to our project, LighGBM provides the importance of explicit features (like numerical data), facilitating understanding of the model's decisions

2 – Summary table

Reference	**Title/source**	**ML Objective**	**Algorithms used**	**Dataset/ features**	**Main results/metrics**
1	*Machine Learning-Based Steam Platform Game's Popularity Analysis* (SCITEPRESS)	Predict game popularity	Log. Regression, SVM, Decision Tree, XGBoost, LightGBM	Steam data including reviews, tags, category …	Best: LightGBM with 88% accuracy and 80% AUC
2	*Deep learning for video game genre classification* (ArXiv 2020)	Automatic classification of video games genres	Deep neural network	Textual features: title, description …	~77% accuracy on average
3	*Prediction and classification of video games* (GrowingScience 2024)	Prediction and classification	GradientBoosting, LogRegression, XGboost, RandomForest, SVM, DecisionTree,	Patterns and trends within a dataset of 4027 video games. Data received from ESRB	Accuracy, Precision, Recall and F1-score used. Best: XGBoost with F1-score pf 89.4%

			BernoulliNB, KNN and AdaBoost		
4	*Steam ML models* (GitHub)	Implementation of many machine learning models on Steam data	LR, GausianNB and classifier version: KNN, RandomForest, MLP	Scrapped steam data	Precision, recall, accuracy, F1-score Best: RandomForest Classifier with accuracy of 81%
5	*Classification and Prediction of Sales Levels* (ResearchGate)	Classification of sales levels on many video games	Naïve Bayes	~13k games and metadata (genre, platforms…)	Accuracy of 71.8%, F1-score of 70%
6	*Prediction and classification of video games* (Medium article)	Machine Learning algorithms for game analysis	DecisionTree Regressor, SVR, RandomForest, Bayes, Lass, XGB, Linear	Kaggle dataset with video games sales and characteristics	RSME and R^2 Train Score, test score, average cv and standard cv
7	*Unveiling Success Drivers in Gaming: A Machine Learning Study Across Steam, Twitch and Metacritic* (Techrxiv)	Analysis of success keys across some platforms	CatBoost, XGBoost, LightGBM, RandomForest, DecisionTree, Multivariate LR	Combination between scraping data and online dataset for Twitch, Steam and Metacritic	MAE, RMSE, NRMSE, R^2
8	*Automatic Classification of Games using Support Vector Machine* (ArXiv)	Classification of games using only text (with SVM)	SVM with LSI for data treatment	iOS App Store dataset	~77% accuracy on average
9	*Contrastive Learning of Generalized Game Representations* (ArXiv)	Training classification neural network with images	ConvNets, ResNet-50 and Supervised Contrastive Learning	Dataset named Sports10 contained 100k experimental results	Training and validation accuracy, silhouette score and t-SNE method for visual results
10	*Video Game Sales Prediction Based on Social Media Data* (IJITCS 2025)	Classification of game age category using text and image combined	Deep neural networks for multimodal processing (image + text fusion, CNN/RNN/Transformers)	Data collected from IGDB (website for open-source game data)	Accuracy

3 - Methods

Our data comes directly from the well-known platform Steam. They represent a complete set of data for the more than 110,000 games present on Steam, with a large choice of data categories on each.
The objective of the project is to predict categories of a game based on is characteristics (metadata), which makes this problem a supervised classification task

The dataset includes, in total, more than 25 different data fields of all types are studied to train the algorithm. There are some grammatical (such as price, average playtime ...) as well as categorical, or binary variable (presence of specific attributes). Before training the models, variables were encoded and normalized (scaled for numerical) when necessary. The target variable corresponds to the game category, making this a multi-class classification problem.
Two machine learning models were selected and evaluated in this project: k-Nearest Neighbours (KNN) and Light Gradient Boosting Machine (LightGBM). These models were chosen because they rely on fundamentally different learning principles, allowing a meaningful comparison between a simple instance-based method and a more advanced approach about trees, which is very adapted to our situation.

- K-Nearest Neighbours (KNN)

KNN is a non-parametric, instance-based learning algorithm. It classifies a new data point by identifying the k closest training samples according to a distance metric (Euclidean distance in this project) and assigning the most frequent class among those neighbours.
KNN was chosen as a basic model because of its conceptual simplicity and its ability to capture local patterns in the data without making strong assumptions about the data links and correlations. However, KNN is very sensitive to feature scaling (due to its method of gathering neighbouring points) and can become expensive for large datasets, as predictions require computing distances to all training points.

- Light Gradient Boosting Machine (LightGBM)

LightGBM is a gradient boosting framework based on decision trees. It builds an ensemble of weak learners in a sequential manner, where each new tree focuses on correcting the errors made by the previous ones. LightGBM introduces optimizations such as histogram-based splitting and leaf-wise tree growth, which make it particularly efficient and scalable.

LightGBM was selected because it is well suited for structured tabular data, handles non-linear relationships effectively, and can naturally manage feature interactions notably when they are minimal or hidden by noise. In addition, it is known to

perform very well on multi-class classification while maintaining a short training time, even on large datasets. Compared to simpler models like KNN, LightGBM is generally more robust to noise and less sensitive to feature scaling.

The choice of using these two algorithms was to compare two complementary approaches. As shown, KNN serves as a simple and interpretable baseline, helping, without having to train an algorithm, to understand how well local correlation links may perform on the problem. On the other hand, LightGBM represents a method known to be robust and effective capable of capturing complex patterns and non-linear relationships in the data.

All experiments were conducted using Python. The main software packages used in this project include:

NumPy and Pandas for data manipulation and preprocessing

Scikit-learn for data splitting, feature scaling, KNN implementation, and evaluation metrics
LightGBM for training and evaluating the gradient boosting model
Matplotlib and Seaborn for data visualization and result analysis
These libraries permit managed, display data in a reliable way, and for some to evaluate machine learning algorithms.

To evaluate the performance of the models, several metrics were used. Accuracy was employed as a primary metric to measure the overall proportion of correctly classified instances. However, since the problem involves multiple classes and class imbalance, additional metrics such as precision, recall, and F1-score were also considered to provide a more comprehensive evaluation.

These metrics allow us for a better understanding of the models' behaviour across different classes and notably ensure that good performance is not achieved at the detriment of poorly predicted categories.

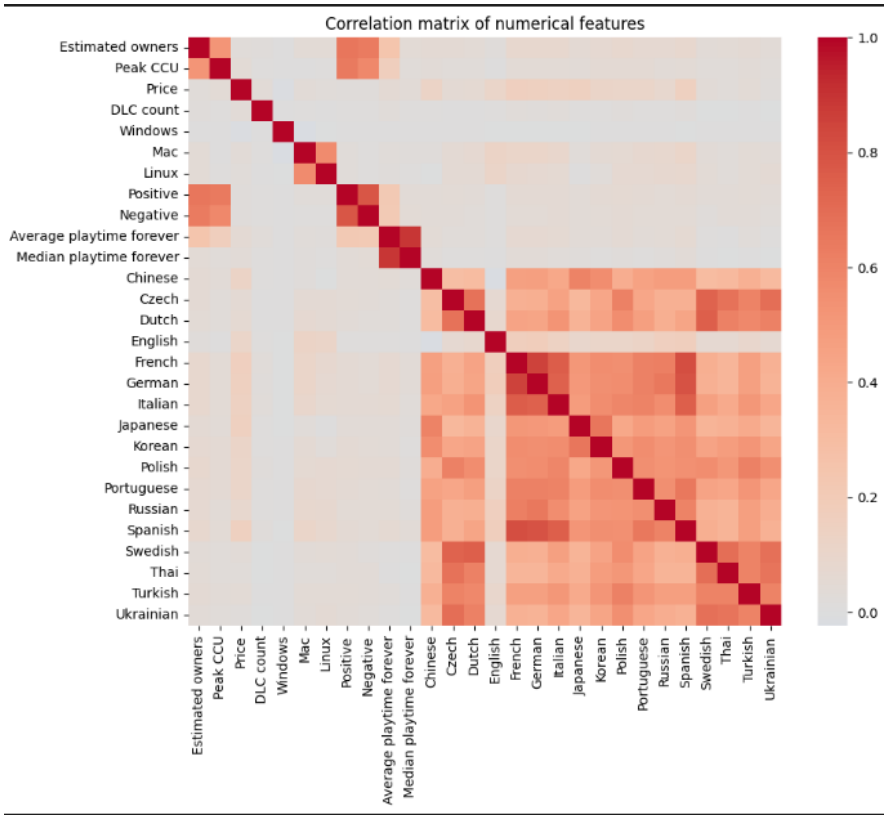
4 - Dataset Preparation

Our dataset is reliable, complete and recently updated. It comes from the website Kaggle, which specializes in the 'open source' of all types of data. The Dataset has more than 240 upvotes, and a usability index of 10/10 (deliver by Kaggle). Some 39 projects and 11 discussions are based on the dataset, proving its usability.

The dataset group information of more than 110,000 gales published on Steam, by 39 variables of all types: numerical, categorical, multi-label, range, binary ...

Once we study the dataset, some variables are too empty values. When this is superior to 30%, the variable is dropped. (like for websites, reviews, support URL ...)
For some other variables, some are not useful for our project (AppID, Discount, Support Email ...) or are too complicated to study (Screenshot, About the game ...) : they are also dropped.

After this step, the dataset retains only the most relevant and complete features. But to make the dataset totally complete, we complete (with meaning approach for numerical or "unknown" word) the raw where there are missing values. Now, the dataset is fully complete. We can analysis these and extract a first correlation matrix:



An initial exploration of the dataset shows that the target variable is highly imbalanced, with some game categories being much more frequent than others. Analysis of numerical and categorical attributes indicates that features such as user score, number of reviews, supported platforms, and release year vary considerably across categories, suggesting their potential predictive value for classification. But I must make it machine-readable to permit the algorithm to view this.

We have identified some problems with format of some columns (like languages), and HTML artefacts, bad separations ... So, we are building advanced cleaning functions to address this.

Firstly, about HTML tag and artefacts, separation of words, removing useless blank and normalization of the letter case. After all of this, the columns language remaining some problems. So, we have built targeted functions for response to some particular case, like correct specific language forms, or duplicate detection (like "Spanish - Spain") But it’s not only the one column which requires specific functions! Like the columns estimated owners, comprising intervals. After, the dataset was clean. But before directly go at Label Encoding, we performed a preliminary label encoding step: for 'Developers' and 'Publishers', named some data as the name 'Independent' (if they are published less than 5 games) for reduce numbers of different data (and so for later, less classes)

This step was also the occasion to rework columns like 'Release Date' (to normalize them) and start a low level of label encoding by converting true/false in 1/0. Also, for columns 'Supported languages', that was pre-processed by retaining only the 20 most frequent languages and then transformed into a binary format using MultiLabelBinarizer, creating one-hot encoded features suitable for machine learning models.

For the columns 'Categories' and 'Genres', we applied a multiclass strategy, retaining only the three dominant classes to limit the number of resulting columns. This avoids having a very large number of different data, and so columns. Once all that is done, we split the data between 80% for train and 20% for test (with sklearn utility)

All categorical features in the training and test sets were transformed into a machine-readable format using One-Hot Encoding, ensuring that each category is represented by a separate binary feature. Additionally, the target labels were encoded using a LabelEncoder to convert them into numerical values suitable for classification models.

But here, there's still a big problem: the classes are totally unbalanced. So, we have applied SMOTE utility for fit resample our train data and have an equal distribution of classes.

Numerical features in the training and test sets were standardized using a StandardScaler to ensure zero mean and unit variance, which helps prevent

features with large ranges from dominating the model. After scaling, dimensionality reduction was applied using Truncated Singular Value Decomposition to reduce the feature space from over 120,000 features to 300 components. This step preserves the most informative aspects of the data while improving computational efficiency. The fitted SVD transformation from the training set was then applied to the test set to ensure consistency. The resulting reduced datasets were used as input for subsequent classification models.

5 - Model Implementation

So, as we already said, we are going to implement two different algorithms: KNN and LightGBM for classifying purposes.

Basic Training

For KNN there is not much to say here because we just train a basic model with the default parameters. Of course, we will proceed hyper parameters tuning after in order to evaluate potential performance improvements and to allow a fair comparison with other models.

For LightGBM, it's a little bit more complex. In fact, we did not start with a purely default model. And that for one several reasons. First, we separated the hyper parameters tuning in two parts for this algorithm:

- The classic research of the best parameters
- The best balance of punishment for the different classes we want to guess

So, we did the first part directly in the basic training stage. In fact, rather than performing an exhaustive search, the algorithm will only make a choice between four sets of parameters that resume basically all the different possibilities suitable for us. Please note that we still checked by RandomizedSearchCV if the parameters we were proposing were valid.

Those sets use only three different parameters in addition to the objective that is, of course "multiclass" and the number of classes that is, of course 3. So those three more parameters are the following:

- **num_leaves**, that controls the number of leaves by tree. More it's high, more complex the model is.
- **learning_rate**, that controls the contribution of each tree. If it's high, the model is quicker but less stable and the opposite is true.
- **n_estimators**, that is the total number of trees in the model. The higher it is the better the model is but bigger the calculations are.

Other parameters like "max_depth" or "min_child_samples" have less impact compared to the tree other, that why we didn't use them here.

And so now, the four sets of parameters are the following:

- 1: "num_leaves": 63, "learning_rate": 0.05, "n_estimators": 600
    - Which is a quick baseline
- 2: "num_leaves": 100, "learning_rate": 0.02, "n_estimators": 1200
    - Which is a deeper model very stable
- 3: "num_leaves": 31, "learning_rate": 0.1, "n_estimators": 400
    - Which is a very aggressive model that is nearly trying to overfit
- 4: "num_leaves": 80, "learning_rate": 0.03, "n_estimators": 800
    - Which is a great deal between the others

So, after selecting the best set of parameters, we are training our basic LightGBM algorithm with it. In our example it's the set number one that was selected the most amount of time.

Remark : LightGBM requires the name of the columns to be in a very strict string chart. So, we encoded them with the name "f_..." and created a map to be able to recover those names in the future.

So now, here are the results for the training of those two algorithms with basic settings on the test dataset :

```
KNN Accuracy: 71.678.

Classification Report
              precision    recall  f1-score   support

           0       0.86      0.79      0.82     11452
           1       0.51      0.55      0.53      2054
           2       0.22      0.34      0.27      1355

    accuracy                           0.72     14861
   macro avg       0.53      0.56      0.54     14861
weighted avg       0.75      0.72      0.73     14861
```

```
LGB Accuracy : 0.8140771145952493

Classification Report
              precision    recall  f1-score   support

           0       0.86      0.92      0.89     11452
           1       0.61      0.59      0.60      2054
           2       0.59      0.23      0.33      1355

    accuracy                           0.81     14861
   macro avg       0.68      0.58      0.61     14861
weighted avg       0.80      0.81      0.80     14861
```

As you can see here there is clearly a better model, which is LightGBM. In fact, while KNN achieves an accuracy of 71%, LightGBM significantly improves the result by achieving an accuracy of 81% which is 10% higher. Moreover, LightGBM outperforms KNN on both macro and weighted F1-score and this indicates that LightGBM not only improves global performance but also achieves a more balanced classification across all classes.

However, if the recall and precision is slightly better with LightGBM, both algorithms achieve a very low recall on the minority classes. That can be explicated by the large difference in the number of supports because the classes are really unbalanced even if we did procced class balancing. That indicates that both algorithms have difficulties guessing right the minority classes whereas they can easily guess the majority class.

Hyper Parameters Tuning

For tuning the KNN algorithm we procced a *RandomizerSearchCv*. We didn't use a grid Search because of the really big size of our dataset. Unfortunately, also due to the size of our dataset we were not able to proceed with an exhaustive tuning. So, the hyperparameter search focused on the most influential parameters for KNN performance.

In particular, we optimized the number of neighbors (*n_neighbors*), which directly controls the bias–variance trade-off of the model. Values ranging from 5 to 24 were evaluated. Moreover, other parameters were fixed to reduce the search space and computation time: the Euclidean distance metric was enforced by setting $p = 2$, and uniform weighting was used to avoid overfitting the model. In fact, we check

the difference between train accuracy and test accuracy in order to determine of the model was underfit or overfit and doing this we saw that other parameters than uniform weighting was always leading to overfitting. Furthermore, the *kd_tree* algorithm was selected to accelerate nearest-neighbor searches in the reduced feature space.

Finally, the RandomizedSearchCV procedure was performed using 3-fold cross-validation and a limited number of iterations (*n_iter = 8*), which allowed us to explore representative configurations while keeping the tuning process computationally time respectable.

For LightGBM now, as we said earlier, this part of hyper parameters tuning was only the process of finding the best punishment factor for the different classes because hyperparameters of the model had already been selected during the basic training phase. So, this tuning step aimed to adjust the decision thresholds in order to achieve a better balance between global performance and minority class detection.

This procedure was implemented by post-processing the class probability outputs of the trained LightGBM model. A punishment factor was applied to the predicted probabilities of the minority classes (classes 2 and 3), in order to increase their relative importance during the final decision step. Several punishment values were evaluated, from 1.0 (no penalization) to 0.7, in order to progressively favor the prediction of minority classes and augment the recall of the algorithm.

So here are the results of both algorithms before and after their tuning :

Before :

```
KNN Accuracy: 71.678.

Classification Report
              precision    recall  f1-score   support

           0       0.86      0.79      0.82     11452
           1       0.51      0.55      0.53      2054
           2       0.22      0.34      0.27      1355

    accuracy                           0.72     14861
   macro avg       0.53      0.56      0.54     14861
weighted avg       0.75      0.72      0.73     14861
```

```
LGB Accuracy : 0.8140771145952493

Classification Report
              precision    recall  f1-score   support

           0       0.86      0.92      0.89     11452
           1       0.61      0.59      0.60      2054
           2       0.59      0.23      0.33      1355

    accuracy                           0.81     14861
   macro avg       0.68      0.58      0.61     14861
weighted avg       0.80      0.81      0.80     14861
```

After :

```
New KNN Accuracy: 71.516.

Classification Report
              precision    recall  f1-score   support

           0       0.86      0.79      0.82     11452
           1       0.51      0.55      0.53      2054
           2       0.22      0.34      0.27      1355

    accuracy                           0.72     14861
   macro avg       0.53      0.56      0.54     14861
weighted avg       0.75      0.72      0.73     14861
```

```
New LGB Accuracy : 0.8001480384900074

Classification Report
              precision    recall  f1-score   support

           0       0.86      0.90      0.88     11452
           1       0.58      0.61      0.60      2054
           2       0.45      0.26      0.33      1355

    accuracy                           0.80     14861
   macro avg       0.63      0.59      0.60     14861
weighted avg       0.79      0.80      0.79     14861
```

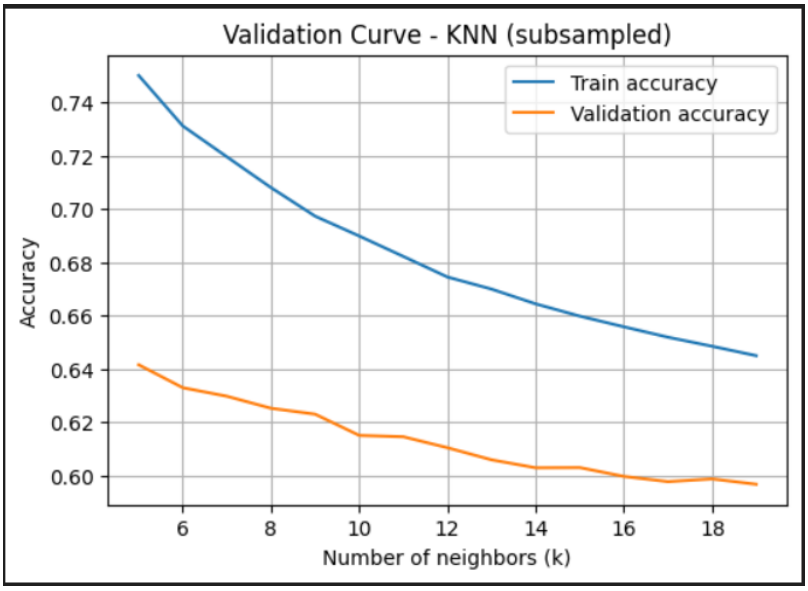
So, as we can see as in the basic training phase, the LightGBM algorithm is clearly better than KNN. And this confirms that, given the size of the dataset and the limited tuning space explored, KNN quickly reaches its performance ceiling.

In fact, KNN nearly didn't change even after the tuning whereas LightGBM had some important changes. First, we can see that its global accuracy had lowered a little, but thanks to that, it's recall for the classes 2 and 3 improve where all the other scores are staying nearly the same.

Better understanding

Validation Curve for KNN :

The validation curve, here, permits us to observe the evolution of the train and test accuracy with the evolution of one of the parameters of KNN. Here, we choose to observe this evolution with the parameter k which is one of the most important parameters in KNN.



As expected, in our situation, the accuracy is heigh for a smaller k (73% of train accuracy for k= 6).

KNN works like that :

- A small k for a more flexible model
- A bigger k for a smoother model

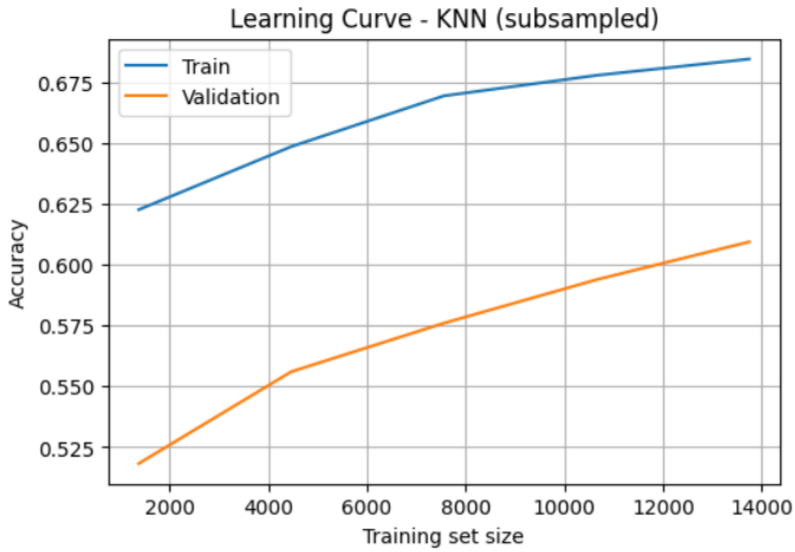
In our situation the model earns nothing more with a bigger k.

Moreover, the gap between train and test accuracy is stable and not too big. In addition, there are no brutal divergences between them so there is no violent overfitting.

So, that indicates that the model needs to stay local and that the close neighbors are informative. In fact, this is due to the reduction of features that we did in data processing. A too high k loses the signal and information.

Learning Curve for KNN :

The learning curve allows us to follow the evolution of accuracy with different sizes of dataset.



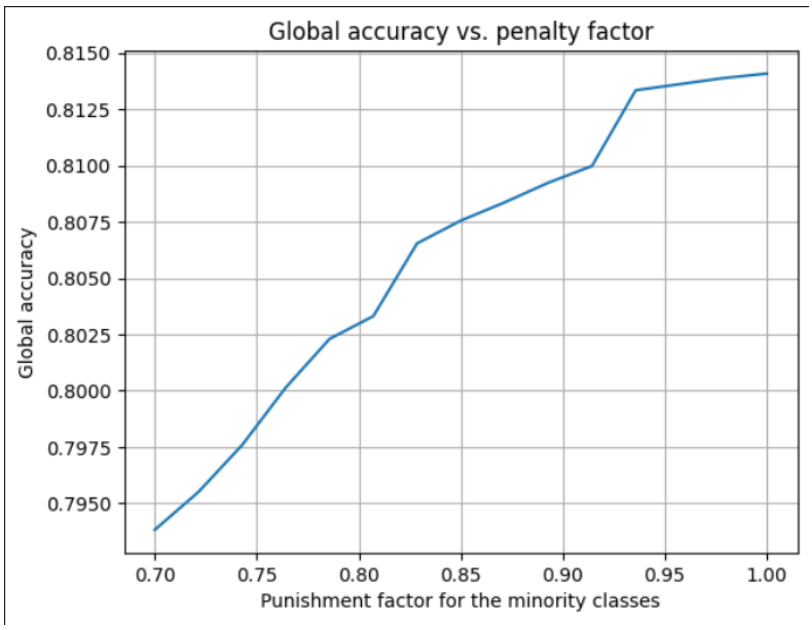
Here, as expected, we can clearly see that the model is better with more data.

Moreover, the distance between those two curves is stable and there is no bigger and bigger distance. That means that there is no high variance.

Nevertheless, both curves are going up but stay at a mean point. None of it converges to high accuracy. That means that there is a moderate bias and that adding more data will help but not a lot.

Global accuracy compared with the punishment factor for LightGBM :

This curve allows us to see the evolution of global accuracy with the evolution of the punishment factor for the minority classes. In fact, reducing the punishment factor increases the relative weight of the minority classes in the final decision rule.



As we can see here, global accuracy increases when the punishment factor increases. Indeed, when p is small, the model is encouraged to predict minority classes more often, which leads to more misclassifications overall and therefore a lower accuracy. As p approaches 1, the prediction rule gets closer to the original LightGBM decision, favoring the majority class and improving overall accuracy. But in our project, we didn't want

our model to just know how to predict the majority class. That's why we privileged a smaller accuracy to increase the prediction of minority classes.

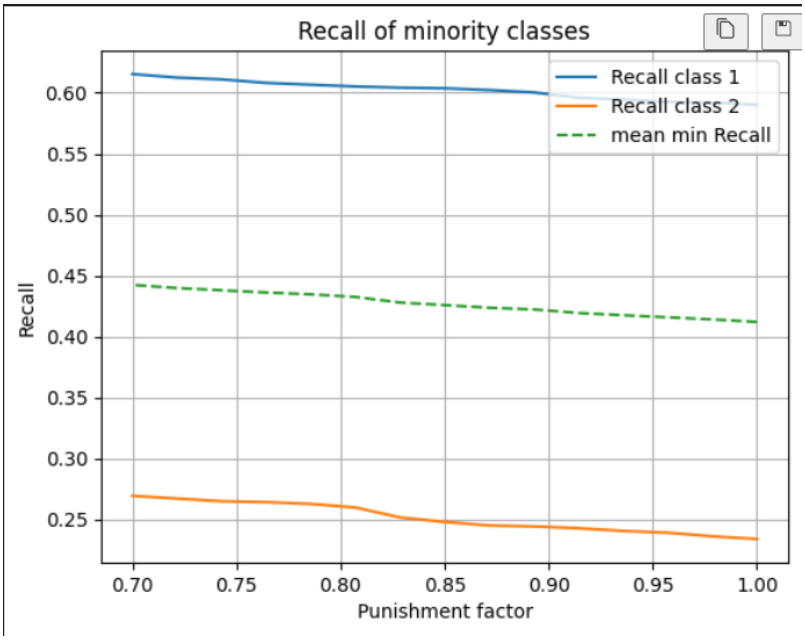
Recall of minority classes

So, as we said, the reduction of the punishment factor increases the recall of the classes and especially of the minority ones.

As you can see here, it's in fact the case. Indeed, when the punishment factor increases the recall decreases.

Unfortunately, the difference is not that big and the gain of reducing punishment factor nearly compensates for the loss of accuracy.

And we can also see that class 2 stays really hard to detect compared to class 1 that are the second majority class.



6 - Model Validation

After building the machine learning models, their performance was evaluated and compared in order to determine their effectiveness and suitability for the category classification task. Model validation was achieved using a test set, which was not used during training, ensuring an unbiased assessment of the models' generalization ability without data leakage.

The KNN (K-Nearest Neighbours) model achieved an overall accuracy of 61.0%, with a weighted F1-score of 65%. While the model performed adequately for first game category, its results varied significantly across classes. This variability suggests that KNN struggled to consistently distinguish between categories with similar feature profiles. Additionally, its reliance on distance-based similarity made it very sensitive to feature scaling and high-dimensional data, which negatively impacted the different classes predictive stability.

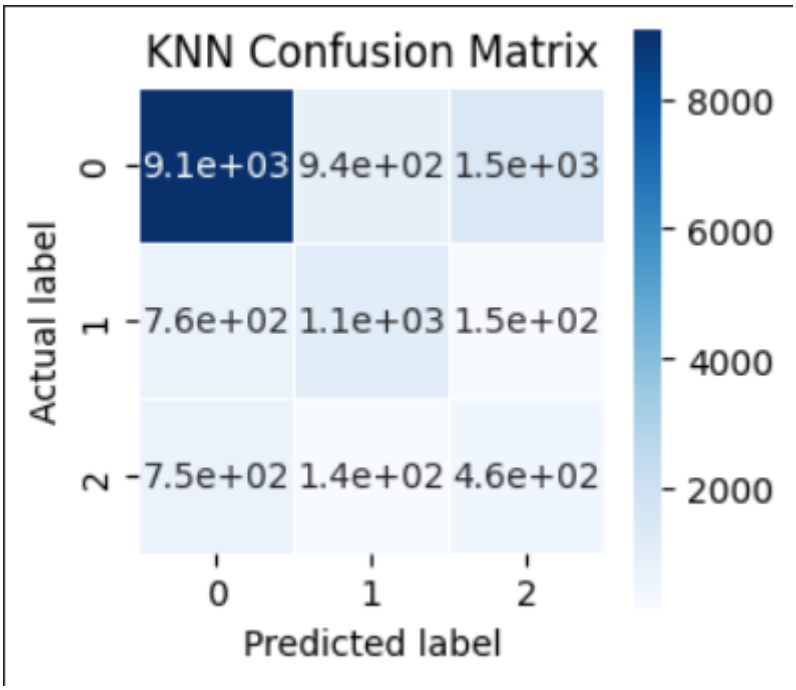
The second algorithm, LightGBM model significantly outperformed KNN across all evaluation metrics. With an accuracy of 84.5% and a weighted F1-score of 82%, indicating a stronger classification performance on all the parameters. LightGBM show a more balanced precision and recall values across multiple classes, highlighting its ability to handle class imbalance and find the complex correlations links between features, however weak they may be. The improvement in F1-score suggests that the model was better at minimizing both false positives and false negatives compared to KNN.

7 - Analysis & Recommendations

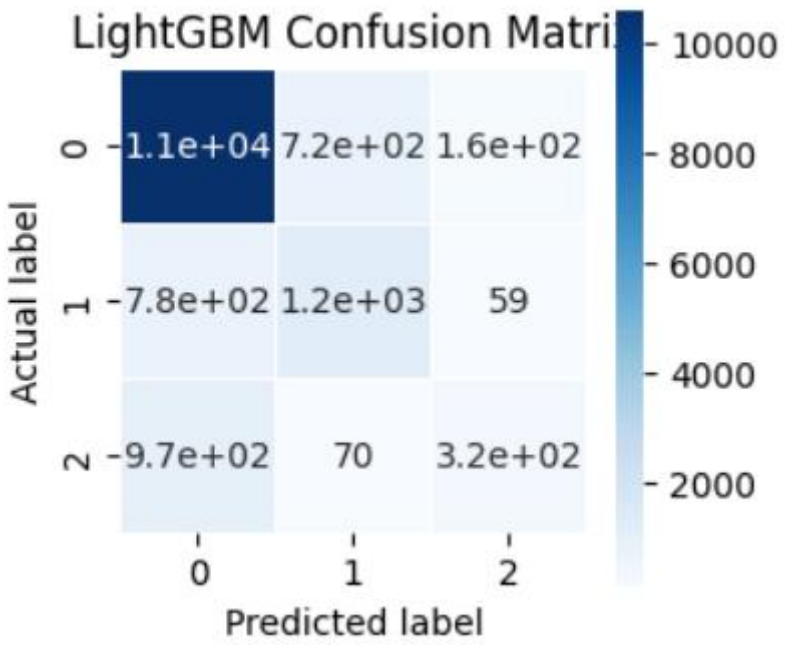
Now, we know globally what's works, and why. But it's time to analysis in depth these results: move beyond raw performance metrics with critically analyse the obtained results.

Because data at the beginning, then data preprocessing, models' choice and their hyper parameters explain well the results; we can't completely understand how the algorithms work, known for each case why a decision was taken but the next analysis information will give you a global idea. This section assesses whether the observed behaviours align with initial expectations, identify anomalies or limitations and compare the findings with some related work presented before.

Firstly, we have some information directly before the training, the confusions matrix. They provide deeper insight into how each model handles class discrimination beyond global performance metrics.

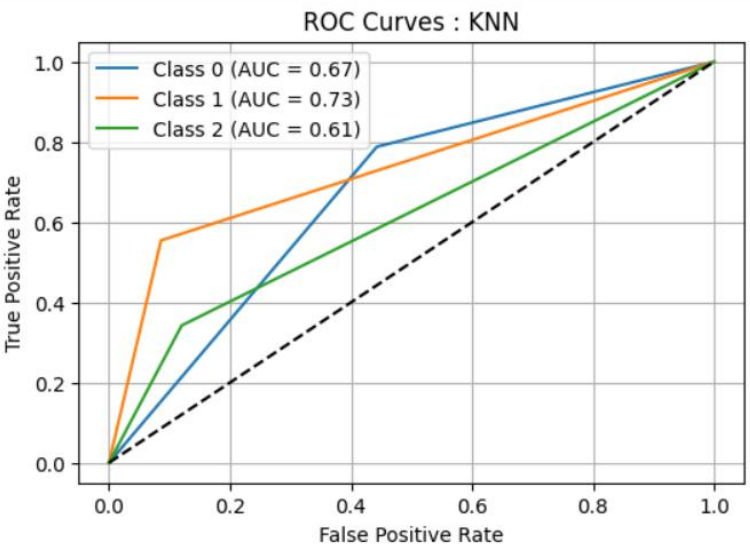


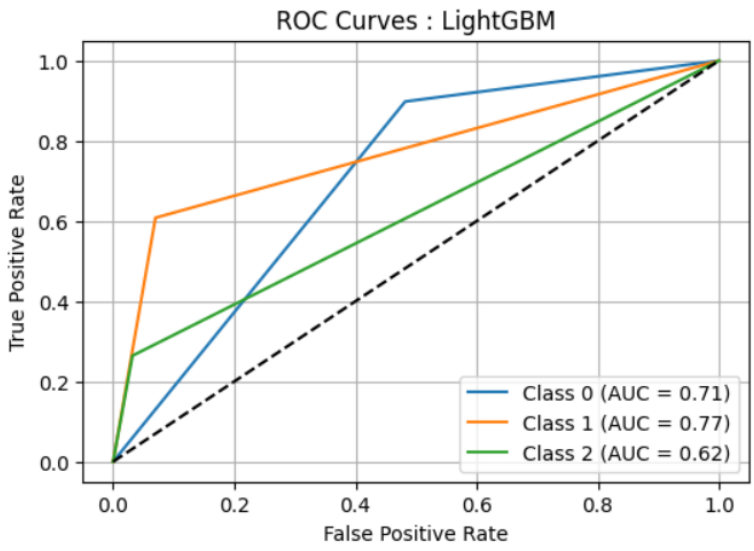
For the KNN model, the confusion matrix reveals widespread misclassification across several game categories, particularly on the classes they possess some overlapping characteristics. Predictions are often distributed across the most represented class (here, the first), indicating that KNN struggles to establish clear decision boundaries in the feature space. This behaviour is consistent with the distance-based nature of the algorithm, which relies heavily on local similarity and becomes unreliable when feature distributions overlap or remain high-dimensional, even after dimensionality reduction.



We view clearly the contrast with the confusion matrix of LighGBM that exhibits a more structured concentrated pattern. There are less errors on the dominant class, that is very well recognized. Globally misclassifications are fewer and tend to occur between categories poorly represented rather than being uniformly spread. This suggests that LightGBM is better able to capture discriminative feature interactions and reduce ambiguity between dominant classes. Although some confusions persist due to intrinsic similarities between certain game categories, LightGBM significantly limits their impact, demonstrating improved class separation and overall robustness compared to KNN.

Just before, we are speaking about model capacity to capture the links between discriminative dominant features. The ROC curves provide additional insight into this capacity ability of both models across individual classes by evaluating the trade-off between the true positive rate and the false positive rate.





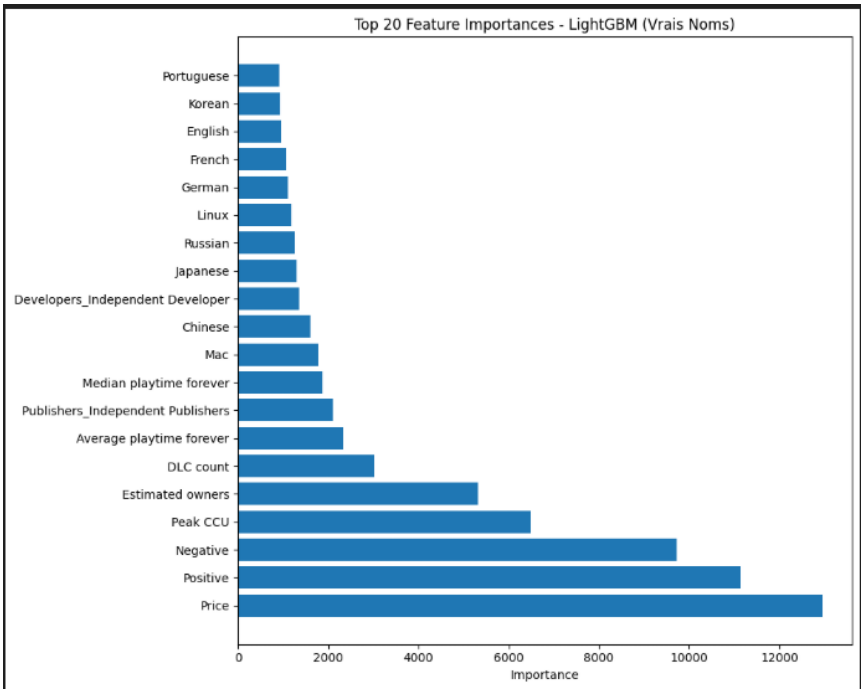
For the KNN model, the ROC curves obtained display weaker separability, with low AUC values for most classes. Although class 1 shows a relatively competitive AUC, the curves for classes 0 and 2 less, notably class 2 is closer to the diagonal, indicating limited discriminative performance. This behaviour highlights KNN's sensitivity to class overlap and its reduced ability to assign confident probability estimates in a multi-class setting.

For the LightGBM model, the ROC curves show consistently higher area under the curve (AUC) values across all classes compared to KNN, indicating a stronger overall capacity to distinguish between positive and negative instances (although the increases are not enormous).

In particular, classes 0 and 1 achieves the highest AUC, suggesting that LightGBM is especially effective at identifying this category, while class 2 remain more challenging due to greater overlap in feature distributions.

Overall, the ROC analysis confirms that LightGBM not only outperforms KNN in terms of accuracy and F1-score but also provides a bit more reliable class-wise discrimination, reinforcing its suitability for this classification task.

For LighGBM, we went further in understanding.



The feature importance analysis provides valuable insight into how the LightGBM model makes its predictions by identifying the variables that contribute most to the classification process. By ranking the top 20 most influential features, this visualization highlights which game attributes play a dominant role in determining the predicted category. This allows for a deeper understanding of the underlying data structure and the relationships learned by the model, beyond aggregate performance metrics.

Therefore, as we might expect, important values such as price, rating, CPU usage, or the number of game processors are highly correlated with the game's category. And in this study, the feature importance plot helps explain why LightGBM is better than a simpler models such as KNN. Because, LighGBM selectively focuses on the most informative feature and their interactions, what's reducing impact of noisy, redundant and weakly correlated variables.

The experimental results obtained in this study are consistent with the trends reported in the related work summarized in Related works part. Studies on video game classification and sales prediction, like [1], [3], [4], [6], predominantly highlight the effectiveness of ensemble-based and boosting algorithms, such as Gradient Boosting and LightGBM, when applied to heterogeneous and high-dimensional game metadata. The best performance achieved by LightGBM in this work aligns with these findings, confirming its ability to model complex non-linear feature interactions and to partially mitigate the effects of class imbalance.

In contrast, simpler distance-based algorithms such as K-Nearest Neighbours are consistently reported in the related works as being sensitive to feature scaling,

dimensionality, and overlapping class distributions. The comparatively weaker performance of KNN observed in our study mirrors these limitations, particularly in a multi-class setting where feature representations of different game categories overlap significantly.

Conclusion

So here we are at the end, with two very different models to compare, but one thing is certain: we chose the right one for our problem (LighGBM).

And this path wasn't straightforward. First, data cleaning required several attempts, detecting errors and artifacts present in the dataset... then finding a way to encode and to make the data machine-readable. So, we went through Label Encoding, which gave us too many columns, then we switched to Multiclass, which was much better but with a considerable loss of information.

But now the data where clean and readable for an algorithm. At that point, we also realized the need for class balancing; otherwise, the two minority classes would be underpowered. And to prevent the algorithms from taking half an hour to execute, dimensionality reduction became mandatory.

Then, it was the train step. One, two, three try ... we should be finding the best hyperparameters for the problems. The HP Tuning stage, with its validations and learning curves, helped us to do this.

And we found a significant improvement: while we were removing all rows with an empty column in the pre-processing step, changing this step to intelligent filling and then randomly removing a third of the rows (so that the algorithms don't take too long to run or crash) was a great benefit for recall especially. So, we are finding an implicit correlation between lines with missing values and the two-minority class.

Then, by showing the graphic of the twenty most important features of LightGBM we view that, in the first six feature, three was date. So, we understand it was an algorithm mistake to mainly based evaluation on a non-representative factor such as dates, although correlations are found. For show, this is our metric before and after columns date was removed.

With date column:

```
KNN With Date
New KNN Accuracy: 61.039.

Classification Report
              precision    recall  f1-score   support

           0       0.86      0.64      0.73     11439
           1       0.40      0.60      0.48      2052
           2       0.16      0.40      0.23      1352

    accuracy                           0.61     14843
   macro avg       0.48      0.55      0.48     14843
weighted avg       0.73      0.61      0.65     14843
```

markdown

```
Light With Date
New LGB Accuracy : 0.843832109411844
              precision    recall  f1-score   support

           0       0.85      0.98      0.91     11439
           1       0.87      0.50      0.63      2052
           2       0.62      0.23      0.33      1352

    accuracy                           0.84     14843
   macro avg       0.78      0.57      0.62     14843
weighted avg       0.83      0.84      0.82     14843
```

markdown

Without date column:

```
New KNN Accuracy: 71.516.

Classification Report
              precision    recall  f1-score   support

           0       0.86      0.79      0.82     11452
           1       0.51      0.55      0.53      2054
           2       0.22      0.34      0.27      1355

    accuracy                           0.72     14861
   macro avg       0.53      0.56      0.54     14861
weighted avg       0.75      0.72      0.73     14861
```

```
New LGB Accuracy : 0.8001480384900074

Classification Report
              precision    recall  f1-score   support

           0       0.86      0.90      0.88     11452
           1       0.58      0.61      0.60      2054
           2       0.45      0.26      0.33      1355

    accuracy                           0.80     14861
   macro avg       0.63      0.59      0.60     14861
weighted avg       0.79      0.80      0.79     14861
```

We can therefore clearly see that without the date column we have an overall positive result, notably for KNN and minority classes for LighGBM.

Now, after all this research, these trials, and the conclusions we've drawn from our mistakes and successes throughout the project, we have a better understanding of how machine learning algorithms work, from the processing of the data we feed them to their evaluation. We have a clearer grasp of their differences, particularly regarding the handling of complex patterns and class imbalances by boosted tree-like algorithms.

And the creation of this report has allowed us to improve our level of algorithm analysis, to better understand the metrics and curves at our disposal, and thus to better judge whether an algorithm is working well or not.

If we were to add to or improve our project, we could continue to refine our hyperparameters with more advanced HP tuning, or, in a completely different way, push the analysis of the data at our disposal by adding deep neural network algorithms and implementing text and image analysis with dimensional frequency matrices (to transcribe text or images into matrices and attempt to merge them on the same dimensions).

Because perfection is never achieved...

Thank for having read this report. We hope you had learnt new things and liked to immerse yourself into our research work

References

[1] : https://www.scitepress.org/publishedPapers/2024/128538/pdf/index.html

[2] : https://arxiv.org/pdf/2011.12143

[3] : https://www.growingscience.com/ijds/Vol8/ijdns_2024_106.pdf

[4] : https://github.com/jsngn/steam-machine-learning/blob/master/steam_models.ipynb

[5] : https://www.researchgate.net/publication/396574427_Classification_and_Prediction_of_Video_Game_Sales_Levels_Using_the_Naive_Bayes_Algorithm_Based_on_Platform_Genre_and_Regional_Market_Data

[6] : https://medium.com/@HannahIgboke/prediction-and-classification-of-video-games-c8bafd86f5e1

[7] : https://www.techrxiv.org/users/860334/articles/1257038/master/file/data/Unveiling%20Success%20Drivers%20in%20Gaming/Unveiling%20Success%20Drivers%20in%20Gaming.pdf?inline=true

[8] : https://arxiv.org/pdf/2105.05674

[9] : https://arxiv.org/pdf/2106.10060

[10] : https://www.sciencedirect.com/science/article/abs/pii/S1875952125000606